BEYOND TOKEN BOUNDARIES: BYTE-LEVEL INFERENCE FOR LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenized language models condition on a discrete encoding of their context, even when a user supplies a prefix that ends inside a token. This mismatch can suppress valid continuations and complicates combining models with different vocabularies. We formulate byte-level inference as conditioning the model's decoded stochastic process on observed bytes. A byte–token representation lemma expresses prefix probabilities as a sum over first-crossing token histories, including tokens that extend beyond the observed prefix. The resulting inference wrapper requires no weight updates and supports probability averaging across heterogeneous vocabularies. We give exact enumeration and incremental update rules, while distinguishing exact marginalization from support-based pruning and finite-beam approximations. On HumanEval Random Span, the reported CodeLlama-7b SPM greedy pass rate increases from 45.0% to 63.9%, exceeding token healing and alignment. Byte-space ensembles reach 65.4% on MMLU and 42.1% on HumanEval. Improvements are not universal: standalone byte decoding sometimes degrades performance, and the GSM8K ensemble trails its strongest member. These results support byte conditioning as a useful interface for boundary-sensitive completion and model combination, with unresolved computational and reproducibility limitations.

1 INTRODUCTION

A language model predicts tokens, but an editor receives a sequence of characters or bytes. These interfaces disagree when a completion prompt ends partway through a token that would appear in the completed text. Encoding the observed prefix commits the model to a token history that need not be a prefix of the encoding of the eventual completion. The model is then asked to continue from an artificial boundary. We call the resulting discrepancy between conditioning on bytes and conditioning on one encoding *tokenization bias*.

This problem is particularly visible in fill-in-the-middle (FIM) code completion. A cursor can occur inside an identifier, so the desired continuation must complete an existing lexical fragment. In the first-order synthetic example, conditioning on token `A` assigns zero probability to another `A`, although the byte process permits this transition with probability $1 - \alpha$. Figure 1 illustrates the analogous completion failure. This is a boundary-conditioning issue; it does not imply that every tokenized model has exact zeros in its output softmax.

We address the interface mismatch without retraining. A token model already induces a distribution over decoded byte strings. Instead of selecting one tokenization of the context, we condition this distribution on the bytes actually observed. Crucially, a token history can explain a byte prefix even if its final token extends beyond the cursor. Summing only token sequences that decode *exactly* to the prefix misses these possibilities.

The same construction provides a shared output space for models with incompatible vocabularies. Once each model predicts the next byte, their conditional probabilities can be averaged directly. This does not guarantee that an ensemble improves on its strongest member, but it removes vocabulary compatibility as a requirement.

Our contributions are threefold. First, we give a representation lemma and proof for byte-prefix probabilities, with explicit assumptions about decoding and termination. Second, we derive reverse

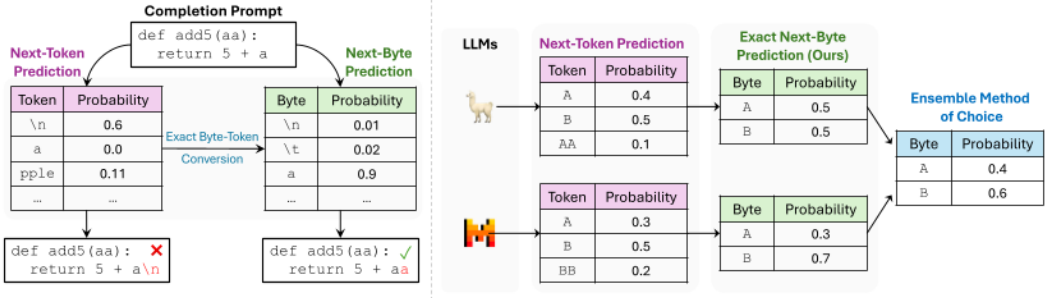


Figure 1: Boundary-sensitive completion and vocabulary-independent combination. Left: the schematic contrasts a fixed token history with conditioning on the observed byte prefix. Right: models with different token vocabularies produce distributions on a common byte alphabet, which can be averaged. The displayed probabilities are illustrative, not benchmark measurements.

enumeration and incremental posterior updates, clarifying when canonical-encoding pruning is exact and when search becomes approximate. Third, we analyze synthetic, infilling, general-language, code-generation, and beam-width results. The strongest observed benefit is boundary-sensitive infilling: SPM greedy performance improves by 18.9 percentage points. Ensemble gains and standalone regressions establish a more qualified empirical picture than universal superiority of byte decoding.

2 Related Work

Subword representations and segmentation ambiguity. Subword tokenization addresses open-vocabulary modeling by representing text through reusable units (Sennrich et al., 2016). Subword regularization explicitly considers multiple segmentations during training (Kudo, 2018). Our construction also recognizes segmentation ambiguity, but operates at inference time on a fixed autoregressive model. Its weights are model probabilities of token histories, rather than probabilities assigned by a separate segmentation sampler. The objective is to recover the decoded model's conditional distribution, not to alter its training objective.

Byte models and completion boundaries. Byte-to-byte models offer a direct byte interface through their pretrained architecture (Xue et al., 2022). Here, the underlying network, vocabulary, and token-level computation remain in place; only the inference interface changes. FIM training enables infilling through transformations of autoregressive training sequences (Bavarian et al., 2022). The present evaluation uses its HumanEval Random Span setting and compares the PSM and SPM prompt orders. Token alignment addresses subword completion through character matching (Athiwaratkun et al., 2024). We instead formulate the target distribution through marginalization over token histories. Token healing is an additional heuristic baseline in the experimental record; its exact implementation is not specified there.

Model and benchmark context. The evaluated general and code models belong to the Llama 2, Code Llama, Mistral, and Yi families (Touvron et al., 2023; Rozière et al., 2023; Jiang et al., 2023; Young et al., 2024). These references describe model families; they do not resolve the checkpoint and naming ambiguities in the experimental record. Evaluation spans MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), GSM8K (Cobbe et al., 2021), HumanEval (Chen et al., 2021), and MBPP (Austin et al., 2021). The experiment-specific scores below come exclusively from the supplied record, rather than from the benchmark or model publications.

3 METHODOLOGY

3.1 THE DECODED MODEL AND THE CONDITIONING MISMATCH

Let $\mathcal{B} = \{0, \dots, 255\}$ and let $\mathcal{V}$ be a finite vocabulary of ordinary tokens. Each token v has a nonempty byte string $d(v) \in \mathcal{B}^+$; decoding concatenates these strings. Let $L = \max_{v \in \mathcal{V}} |d(v)|$. We require a lossless, concatenative byte representation: textual cleanup or context-dependent decoding cannot be silently substituted for d. A fixed initial token history, such as a beginning-of-sequence marker, is implicit throughout. The autoregressive model defines

$$P(t_{1:k}) = \prod_{i=1}^{k} P(t_i \mid t_{1:i-1}), \qquad P(\epsilon) = 1. \tag{1}$$

Termination is represented separately by `EOS`. The ordinary-token and termination probabilities sum to one. Other control symbols must be fixed as part of the prompting protocol, not treated as empty byte tokens in the enumeration.

Write $x \preceq y$ when x is a byte prefix of y, and $x \prec y$ for a strict prefix. If X is the decoded random output, define the *prefix mass*

$$Q(x) = \Pr(x \preceq X), \qquad q(b \mid x) = \frac{Q(xb)}{Q(x)}, \quad b \in \mathcal{B}, \tag{2}$$

for $Q(x) > 0$. Thus $Q(x)$ is not the probability that the output ends at x. The remaining conditional probability is termination at x.

For a deterministic encoder E, ordinary token continuation from $E(x)$ induces a potentially different byte distribution. Unlike conditioning on $x \preceq X$, it fixes both the token history and a token boundary at the end of x. Tokenization bias is the difference between these conditioning operations. Exact conversion recovers q, the distribution induced by the *trained model*; it does not generally recover the unknown data-generating distribution.

3.2 THE BYTE–TOKEN REPRESENTATION LEMMA

For nonempty $x = x_{1:n}$, define the set of *cover encodings*

$$\mathcal{C}(x) = \big\{t_{1:k} : d(t_{1:k-1}) \prec x \preceq d(t_{1:k})\big\}. \tag{3}$$

A cover is the shortest token history on its path whose decoded length reaches the end of x. Its final token may end at x or extend past it. Set $\mathcal{C}(\epsilon) = \{\epsilon\}$.

Lemma 1 (Byte–token representation)**.** *Under the decoding assumptions above, for every finite byte string x,*

$$Q(x) = \sum_{t \in \mathcal{C}(x)} P(t). \tag{4}$$

Consequently, substituting these sums into Equation 2 gives exact next-byte probabilities whenever $Q(x) > 0$.

Proof. For a token trajectory whose decoding begins with x, there is a unique first token at which the decoded length reaches $|x|$, since ordinary tokens have positive byte length. Its history is in $\mathcal{C}(x)$. Conversely, every trajectory beginning with such a history decodes to a string beginning with x. These token-prefix events are disjoint: one cover cannot strictly extend another cover of the same x. Their union is precisely the event defining $Q(x)$, and each has probability $P(t)$. For the empty string, the identity is $Q(\epsilon) = 1$. □

The distinction between exact encodings and covers is essential. With tokens decoding to `A`, `B`, and `AA`, the covers of prefix `A` include both (`A`) and (`AA`). The latter already determines the next byte to be `A`, without requiring another token prediction. Conditioning only on token (`A`) discards that probability mass.

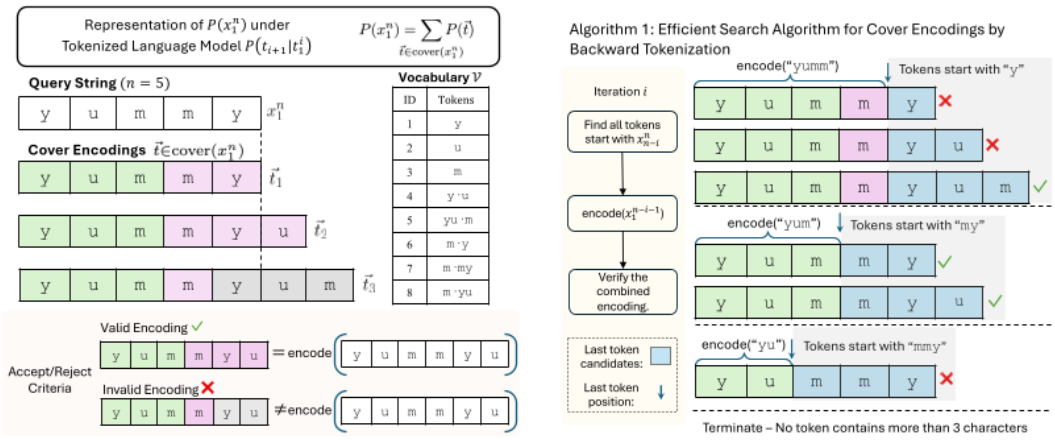


Figure 2: Cover encodings and backward candidate search with a BPE tokenizer. A cover can extend beyond the observed string. The displayed accept/reject tests express canonical-tokenizer compatibility; interpreting rejection as zero probability additionally requires the model-support condition discussed in Section 3.3.

3.3 Valid Encodings, Reverse Search, and Exactness

Deterministic tokenizers select a canonical encoding. A token sequence t is *canonical* for its decoded string if $E(d(t)) = t$; other sequences are noncanonical for that string. This is a property of the tokenizer, not a guarantee about $P(t)$. A trained model can assign positive mass to noncanonical sequences. Moreover, pruning a partial history requires ruling out all compatible future extensions, since a canonical encoding may change at its trailing boundary.

Figure 2 illustrates the proposed canonical-validation strategy. It is exact only under an additional support condition: every discarded token-prefix event must have zero model probability. Without that condition, Lemma 1 requires retaining every compatible history with positive mass. Renormalizing after removing nonzero mass defines a different model. Our general enumeration therefore does not assume canonical pruning is safe.

Reverse search limits the *last token's* start position to a window of L bytes. For each possible preceding boundary j, enumerate token histories decoding exactly to $x_{1:j}$, and attach each token that covers the remaining observed suffix. Define

$$F(0) = \{\epsilon\},$$
$$F(j) = \bigcup_{\substack{v \in \mathcal{V}:\ |d(v)| \le j, \\ d(v) = x_{j-|d(v)|+1:j}}} \{s \,\|\, v : s \in F(j - |d(v)|)\}. \tag{5}$$

Here $\|$ appends a token. These sets contain token identities and complete histories, not merely decoded strings.

Algorithm 1 (Exact reverse enumeration of covers)**.** Given $x_{1:n}$ with $n > 0$, a vocabulary decoder, and the model:

1. Initialize $\mathcal{C}(x) = \varnothing$ and construct the history sets $F(j)$ using Equation 5, caching shared token prefixes where possible.
2. For each $j = \max(0, n - L), \ldots, n - 1$, find every v such that $|d(v)| \ge n - j$ and $x_{j+1:n} \preceq d(v)$.
3. For every $s \in F(j)$ and matching v, add $s\|v$ to $\mathcal{C}(x)$ and evaluate $P(s\|v)$ using Equation 1. Discard only histories with certified zero probability.
4. Sum these weights to obtain $Q(x)$. Use the posterior update below to compute the next-byte distribution and continue decoding.

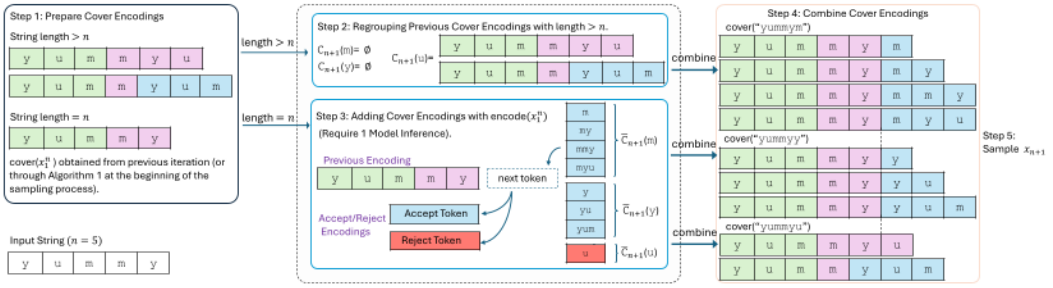


Figure 3: Incremental cover update. Covers extending beyond the cursor contribute their residual next byte; covers ending at the cursor require expansion by next-token probabilities. The drawing suppresses history probability factors, which are required by Equations 6 and 7. Canonical rejection is subject to the support qualification in Section 3.3.

All loops are finite because ordinary tokens consume at least one byte. Vocabulary tries can index matching token prefixes and suffixes; cached model states avoid recomputing identical token histories. However, distinct histories with identical decoded bytes generally have different Transformer states and cannot be merged into one state. The number of histories can grow exponentially with context length. Reverse search bounds the terminal overlap, not the total cost, and the experimental record supplies no latency or memory measurements establishing practical efficiency.

3.4 Incremental byte prediction

Normalize cover weights to obtain the posterior

$$\pi_x(t) = P(t)/Q(x), \qquad t \in \mathcal{C}(x). \tag{6}$$

For $n = |x|$, each cover contributes either a known residual byte or a next-token distribution:

$$K_t(b) = \begin{cases} \mathbf{1}\{d(t)_{n+1} = b\}, & |d(t)| > n, \\ \sum_{v \in \mathcal{V}: d(v)_1 = b} P(v \mid t), & |d(t)| = n. \end{cases} \qquad q(b \mid x) = \sum_{t \in \mathcal{C}(x)} \pi_x(t) K_t(b). \tag{7}$$

For a boundary cover, the termination kernel is $P(\texttt{EOS} \mid t)$; for an overhanging cover it is zero. Including this outcome makes the distribution normalized even for models that can terminate.

After choosing byte b, keep each overhanging cover whose next residual byte equals b, with its existing unnormalized weight. Expand each boundary cover t by every token v beginning with b, assigning weight $P(t)P(v \mid t)$. These histories form $\mathcal{C}(xb)$; normalize by $Q(xb)$. Model evaluations are needed only for boundary covers, while overhanging covers reuse existing weights. Figure 3 illustrates this regrouping. Log probabilities and log-sum-exp provide a direct numerically stable implementation.

Sampling from these conditionals, including termination, preserves the decoded model distribution. Byte-greedy decoding instead chooses $\arg\max_b q(b \mid x)$; it need not reproduce token-greedy decoding. Thus a greedy-score change can result from a different decision rule as well as different boundary conditioning. Truncating posterior histories to a finite beam is an approximation unless the discarded mass is zero. The reported beam-width study does not specify whether the beam retains latent token histories or candidate output continuations, so it cannot establish convergence to the exact distribution.

3.5 FIM contexts and heterogeneous ensembles

For FIM, the control markers specifying prefix, suffix, and middle must retain their token-level role. Byte marginalization applies to the text between fixed controls; ordinary tokens cannot cross a control marker. A tokenizer-specific implementation must specify which observed text segments

are marginalized and which preceding token history is held fixed. Simply grouping the logits after one encoding of the complete prompt does not perform the marginalization in Lemma 1.

For models $m = 1, \ldots, M$ observing a common byte context with corresponding control protocols, let q_m be their converted distributions. The context must have positive prefix mass under every included model. With fixed weights $\lambda_m \geq 0$ summing to one, define

$$q_{\text{ens}}(b \mid x) = \sum_{m=1}^{M} \lambda_m q_m(b \mid x). \tag{8}$$

The experiments use two models with equal weights. Every member updates its own cover posterior after each shared output byte. Termination must be aligned as a shared outcome. This is an average of next-byte conditionals, which defines an autoregressive ensemble; it is generally different from selecting one model once and sampling its entire output. Vocabulary sizes and token identities need not match, but unequal member quality can still impair performance.

4 Experiments

4.1 Setup and scope of the evidence

The evaluation covers three general-purpose models (Llama2-7B, Yi-1.5-6B, and Mistral-7B-v0.3), code-specialized models, and synthetic first- and third-order Markov processes. A six-hidden-layer GPT-2 architecture is also listed in the synthetic setup. No model weights are updated for byte conversion. General-language evaluations report accuracy; code evaluations report the logged greedy, @10, or @1 pass rates. All table entries are percentages, and all differences in the text are percentage points unless stated otherwise. A dash denotes an unreported configuration.

For infilling, CodeLlama-7b is evaluated on HumanEval Random Span under Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM) prompting. Baselines are token decoding, token healing, and token alignment. The general-language suite comprises MMLU, PIQA, Natural Questions (NQ), TriviaQA, and GSM8K; code generation uses HumanEval and MBPP. Ensembles average the byte probabilities of the top two models per task, as described in the record. The record does not specify whether this selection used held-out data. Voting results are available only for MMLU and PIQA, without a detailed tie-breaking protocol.

The record omits random seeds, evaluated example counts, confidence intervals, exact prompts, checkpoint revisions, most sampling settings, and code-execution details. We therefore treat the scores as descriptive point estimates and do not assert statistical significance. The @10 label is retained from the record; the supplied infilling figure describes it as the maximum over a temperature sweep, rather than performance at one fixed reported temperature. Further reproducibility gaps are listed in Appendix C.

4.2 Synthetic transition recovery

The first-order experiment provides a controlled boundary failure: after token `A`, the token-space transition to `A` is zero, whereas the generating byte chain assigns probability $1 - \alpha$. Figure 4 shows the transition diagram and third-order comparison. The latter reports that the correction recovers the conditional next-byte probabilities across the eight displayed three-byte histories, while the uncorrected estimate can depart substantially from the ground truth.

These observations support the proposed mechanism in a controlled setting. They do not establish a general guarantee of recovery of a real training distribution: the lemma guarantees equivalence to the decoded model, and matching the generating chain additionally requires that the learned model represent it sufficiently well. Neither transition parameter values, training sizes, nor quantitative recovery errors are provided, so we do not infer an error tolerance from the plotted agreement.

4.3 Code infilling and prompt order

Table 1 shows the clearest benefit. Token-level greedy performance falls from 64.9 under PSM to 45.0 under SPM, a 19.9-point gap. Byte prediction raises SPM greedy performance to 63.9, a gain of

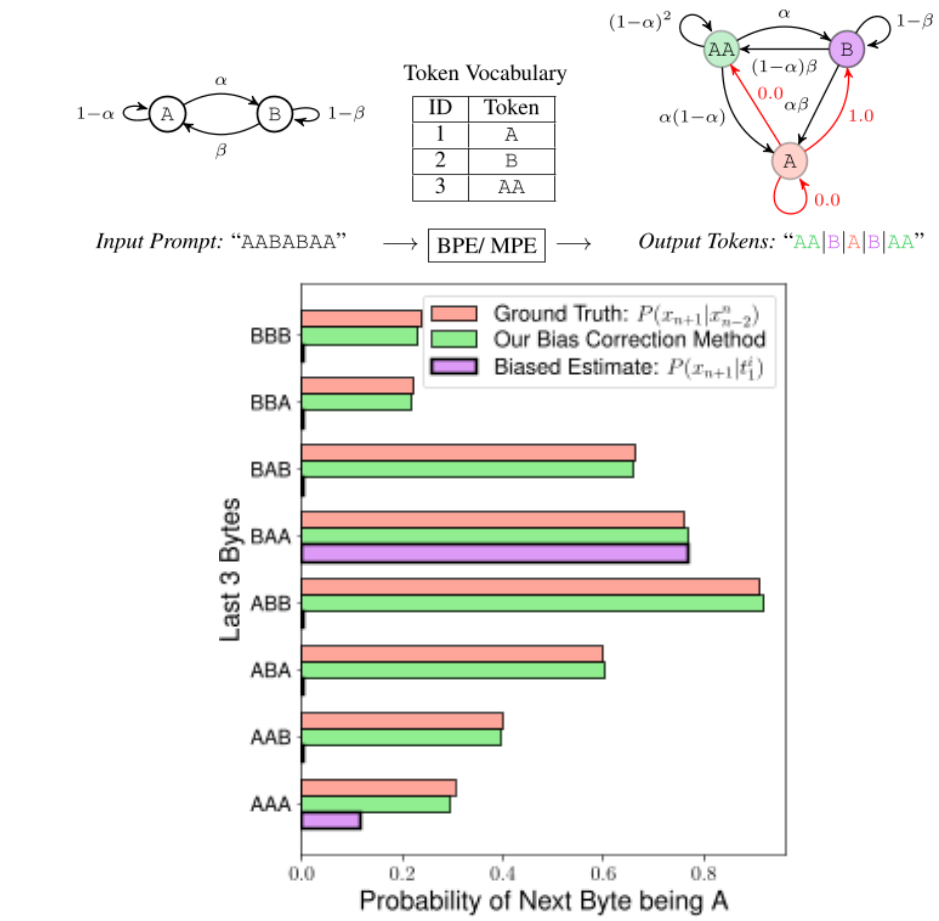


Figure 4: Synthetic evidence for the boundary-conditioning mismatch. Top: the first-order chain and vocabulary illustrate the zero transition from token A to A, despite a byte-level transition probability of $1 - \alpha$. Bottom: the third-order plot compares ground truth, corrected predictions, and biased estimates for the next byte being A. Numerical error estimates and uncertainty are not available.

Table 1: HumanEval Random Span pass rates (%) for CodeLlama-7b. The @10 columns retain the temperature-sweep reporting convention of the supplied figure. Dashes indicate configurations not reported.

	PSM		SPM	
Method	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
Token-level + healing	–	–	62.7	83.8
Token-level + alignment	–	–	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

18.9 points, and reduces the PSM–SPM gap to 1.8 points. SPM @10 increases from 66.5 to 84.3, a gain of 17.8 points. These absolute differences give a precise interpretation of the log's approximate improvement statement.

Against boundary-repair heuristics, byte decoding improves SPM greedy performance by 1.2 points over healing and 0.9 over alignment; the corresponding @10 gains are 0.5 and 1.4 points. These small margins lack uncertainty estimates. Qualitative cases in the record involve the identifiers `delimeter`, `collatz`, and `palindrome`, where a cursor divides a word. The reported success-

Table 2: Reported general-language accuracy (%). Each model has token (T) and byte (B) rows. The ensemble averages the top two models in byte space. Bold denotes the highest reported score per benchmark across both decoding modes.

Model	Mode	MMLU	PIQA	NQ	TriviaQA	GSM8K
Llama2-7B	T	45.4	78.1	25.0	58.4	15.1
	B	45.7	78.2	23.4	58.1	12.5
Yi-1.5-6B	T	63.4	78.5	22.8	53.7	61.3
	B	63.4	78.5	22.7	53.4	**61.5**
Mistral-7B-v0.3	T	62.1	80.1	28.5	63.6	39.0
	B	62.1	80.3	28.8	63.5	39.0
Voting (top-2)	–	62.1	80.1	–	–	–
Byte ensemble	B	**65.4**	**80.7**	**30.0**	**64.2**	55.8

Table 3: Reported code-generation @1 pass rates (%). Model names follow the code-generation log, including its "CodeLlama2" labels; equivalence to the FIM checkpoint is not established. Bold denotes the highest reported result per benchmark.

	HumanEval @1		MBPP @1	
Model	Token	Byte	Token	Byte
CodeLlama2-7b	32.3	28.7	40.6	42.4
CodeLlama2-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 byte ensemble	–	**42.1**	–	**53.6**

ful byte continuations are consistent with the boundary mechanism, but no per-example traces are provided.

Byte decoding also improves PSM greedy performance by 0.8 points and PSM @10 by 0.1. Because byte-greedy and token-greedy rules differ, these small PSM gains do not independently isolate tokenization bias. The supplied temperature-sweep figure is reproduced in Appendix B for context.

4.4 General-language models and byte-space ensembles

Standalone conversion has mixed effects (Table 2). Llama2-7B improves slightly on MMLU and PIQA but falls by 1.6 points on NQ and 2.6 on GSM8K. Yi-1.5-6B is unchanged on MMLU and PIQA, improves by 0.2 on GSM8K, and declines slightly on NQ and TriviaQA. Mistral changes by at most 0.3 points on these tasks. These outcomes do not support a general claim that an exact byte interface improves task accuracy.

The ensemble exceeds the strongest reported individual result, considering both decoding modes, by 2.0 points on MMLU, 0.4 on PIQA, 1.2 on NQ, and 0.6 on TriviaQA. It also exceeds the reported voting baseline by 3.3 points on MMLU and 0.6 on PIQA. These comparisons demonstrate useful combinations of model predictions, subject to the unresolved member-selection protocol.

GSM8K supplies a clear counterexample: the ensemble scores 55.8, below Yi-1.5-6B's byte score of 61.5 by 5.7 points. The two strongest standalone models differ substantially here, with Yi at 61.5 and Mistral at 39.0 in byte mode. Uniform probability averaging can dilute a stronger model's predictions when another member is less capable or differently calibrated. The record does not isolate the cause, so capability imbalance is a possible explanation rather than a demonstrated causal result.

4.5 Code generation and beam width

On HumanEval, all three standalone models decline under byte decoding: by 3.6, 2.3, and 1.9 points, respectively (Table 3). Nevertheless, the ensemble reaches 42.1, exceeding the strongest token result

Table 4: Reported byte-level FIM pass rates (%) by beam width. Width one matches the greedy byte entries in Table 1. The record does not specify the beam's search states or scoring rule.

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	67.7	66.8

of 38.4 by 3.7 points and the strongest standalone byte result of 36.5 by 5.6 points. The qualitative record identifies Yi-Coder-1.5B and CodeLlama2-13b as the HumanEval ensemble members. On MBPP, byte decoding improves the 7b model and Yi-Coder while slightly reducing the 13b model's score. The ensemble ties Yi-Coder's byte score of 53.6; it does not improve on that member.

Increasing beam width from one to four raises the reported FIM score by 2.0 points in PSM and 2.9 in SPM (Table 4). The intermediate width also improves both modes. This supports additional search as a useful empirical choice in the reported setup, but does not quantify retained probability mass, decoding latency, or approximation error. It cannot establish that wider beams recover the exact marginal.

4.6 Limitations and interpretation

Theoretical equivalence and task performance address different questions. The lemma is exact for the decoded model under the stated assumptions; a wrong or poorly calibrated model remains wrong or poorly calibrated after conversion. Canonical filtering, finite beams, and tokenizer preprocessing require separate validation. Byte-greedy decisions can change outputs even when both procedures use the same underlying stochastic model.

The reported results are most persuasive for an interface failure that is deliberately exposed by the SPM setting. They provide weaker evidence for general improvements in reasoning or code generation, where multiple regressions occur. Search cost, statistical uncertainty, checkpoint identity, and ensemble-selection methodology remain unresolved. Consequently, the record supports an inference formulation and promising empirical behavior, rather than a fully audited claim of efficient, universally superior token-free modeling.

5 Conclusion

Conditioning a tokenized model on observed bytes requires marginalizing first-crossing token histories, including tokens that extend beyond the prompt. The resulting byte interface preserves the decoded model distribution under explicit support and decoding assumptions and permits probability averaging across different vocabularies. Reported experiments show substantial recovery of SPM infilling performance and useful ensemble gains, alongside standalone regressions and a weaker GSM8K ensemble. Future work should measure retained mass and computational cost, resolve tokenizer-specific pruning conditions, and evaluate ensemble selection and uncertainty under fully specified protocols.

Reproducibility statement

The paper gives a proof, exhaustive reverse enumeration, and incremental probability updates for the stated abstract model. All reported numbers are transcribed from the supplied experimental record; the figures are supplied artifacts. Appendix C identifies missing implementation and evaluation details. The available materials do not include runnable experiment code, complete prompts, or sufficient metadata for independent numerical reproduction. No additional experiments were conducted for this writeup.

REFERENCES

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Robert Kwiatkowski, Ramesh Nallapati, Parminder Bhatia, and Bing Xiang. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 15725–15738, 2024. doi: 10.18653/v1/2024.findings-acl.929. URL https://aclanthology.org/2024.findings-acl.929/.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022. URL https://arxiv.org/abs/2207.14255.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, 2020. doi: 10.1609/aaai.v34i05.6239.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023. URL https://arxiv.org/abs/2310.06825.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018. doi: 10.18653/v1/P18-1007. URL https://aclanthology.org/P18-1007/.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, et al. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019. doi: 10.1162/tacl_a_00276.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023. URL https://arxiv.org/abs/2308.12950.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016. doi: 10.18653/v1/P16-1162. URL https://aclanthology.org/P16-1162/.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. URL https://arxiv.org/abs/2307.09288.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. doi: 10.1162/tacl_a_00461.

Alex Young, Bei Chen, Chao Li, Chengen Huang, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024. URL https://arxiv.org/abs/2403.04652.

A FURTHER CONSEQUENCES OF THE REPRESENTATION LEMMA

A.1 COMPLETENESS OF REVERSE ENUMERATION

For any $t_{1:k} \in \mathcal{C}(x_{1:n})$, let $j = |d(t_{1:k-1})|$. By definition, $j < n$ and $n - j \leq |d(t_k)| \leq L$, so $j \in [\max(0, n - L), n - 1]$. The preceding history belongs to $F(j)$ and the last token passes Algorithm 1's prefix test. Thus every cover is enumerated. Conversely, each enumerated history decodes to $x_{1:j}$ followed by a token covering $x_{j+1:n}$, and therefore satisfies Equation 3. Positive token lengths make the recurrence for $F(j)$ acyclic. Distinct token IDs must remain distinct even if their decoded byte strings coincide.

A.2 NORMALIZATION AND STOPPING

Let $R(x)$ denote the probability that the decoded output terminates exactly at x. Every trajectory beginning with x either stops there or emits a next byte. Hence

$$Q(x) = R(x) + \sum_{b \in \mathcal{B}} Q(xb), \qquad R(x) = \sum_{\substack{t \in \mathcal{C}(x): \\ |d(t)| = |x|}} P(t) P(\texttt{EOS} \mid t). \tag{9}$$

The next-event distribution comprises $q(b \mid x)$ and $R(x)/Q(x)$. If termination is masked for a fixed-length task, probabilities must be conditioned on continuation explicitly; otherwise masking changes the process. A prefix with $Q(x) = 0$ has no well-defined conditional distribution under this model. Choosing a fallback for such a prefix is an additional modeling decision, not a consequence of the lemma.

For a finite continuation $y_{1:r}$ before termination, the conditional byte probabilities telescope:

$$\prod_{i=1}^{r} q(y_i \mid x y_{1:i-1}) = \frac{Q(x y_{1:r})}{Q(x)}. \tag{10}$$

Including the final stopping probability recovers the conditional probability of that finite decoded output. This establishes statistical equivalence for sampling, without asserting equivalence of greedy decisions.

A.3 A CONDITIONAL BOUND FOR POSTERIOR TRUNCATION

Suppose a subset $S \subseteq \mathcal{C}(x)$ is retained and its exact posterior mass is $1 - \delta$, with $0 \leq \delta < 1$. Let q_S be the distribution obtained by renormalizing these exact retained weights and using the exact kernels, including termination. If $\delta > 0$, write q_D for the analogous distribution of discarded histories. Then

$$q = (1 - \delta) q_S + \delta q_D, \qquad \mathrm{TV}(q, q_S) = \delta \, \mathrm{TV}(q_D, q_S) \leq \delta. \tag{11}$$

For $\delta = 0$ the distributions coincide. This bound is local to a context and requires the actual discarded posterior mass. Beam width alone does not determine δ, and accumulated approximation errors require additional analysis. The experimental record contains no retained-mass measurements, so this bound supplies no numerical guarantee for Table 4.

B SUPPLEMENTARY INFILLING VISUALIZATION

Predictions	PSM Greedy	PSM @10	SPM Greedy	SPM @10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

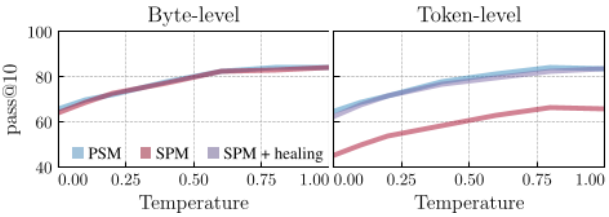


Figure 5: HumanEval Random Span comparison for CodeLlama-7b. The table reproduces the values in Table 1; the plots show the recorded temperature sweep. The figure explicitly defines @10 as a maximum over that sweep. Raw per-temperature measurements, sample counts, and uncertainty estimates are not provided, so intermediate curve values are not treated as additional numeric results.

C DETAILS NEEDED FOR NUMERICAL REPRODUCTION

The mathematical algorithm specifies a reference target for an implementation, but it does not reconstruct missing details of the experiments. Independent reproduction requires the following information beyond the supplied record:

1. **Models and tokenizers.** Exact checkpoint identifiers and revisions; special-token handling; the lossless byte decoder; normalization and whitespace rules; maximum token byte length; and whether the "CodeLlama2" code-generation labels denote the same family or checkpoints as the FIM model.
2. **Inference wrapper.** Which prompt segments are marginalized; the initial fixed token history; the canonical-validity test and its support justification; retained history counts; cache management; stopping rules; and any search approximation. A finite softmax alone does not certify zero mass for a tokenizer-invalid path.
3. **Synthetic experiments.** Complete first- and third-order transition tables, α and β, tokenizer construction, training corpus sizes, optimizer settings, GPT-2 configuration beyond its six layers, seeds, and numerical probability-recovery errors. The record does not link each plotted result to a fully specified trained configuration.
4. **FIM and code evaluation.** Exact PSM/SPM templates and control IDs; random-span selection; benchmark split and number of examples; healing and alignment implementations; temperature grid; number of samples; pass@k estimation; generation limits; and the execution harness. The @10 sweep maximum requires a reported selection protocol. The beam state space, ranking objective, normalization, and stopping criterion must also be specified.
5. **General-language and ensemble evaluation.** Benchmark versions and splits; few-shot examples; answer-scoring, extraction, and normalization rules; prompts; task-specific top-two membership; member-selection data; voting and tie-breaking rules; and whether stopping probabilities are averaged. Multiple-choice and free-generation accuracies cannot be reconstructed from one shared unspecified scoring rule.
6. **Uncertainty and cost.** Per-example outcomes or repeated runs for uncertainty estimates, hardware and software versions, latency, memory, and token-model calls per generated byte. These are needed to assess the practical benefit of reverse search and wider beams.